Proceedings of the Fifth International Conference on Communication and Electronics Systems (ICCES 2020)
IEEE Conference Record # 48766; IEEE Xplore ISBN: 978-1-7281-5371-1

Validity of Forensic Evidence using Hash Function

Pradeep K C (Student)
Department of computer science, PES University, Bangalore
Email: kcpradeep05@gmail.com

Rajashree Soman (Asst. Professor)
Department of computer science, PES University, Bangalore
Email: rajashrees@pes.edu

Prasad Honnavalli (Professor)
Department of computer science, PES University, Bangalore
Email: prasadhb@pes.edu

***Abstract*—In any crime scene, the cyber forensics team look out for various evidences. In particular, they seek digital evidences such as surveillance system, hard disk drives, pen drives, and so on. After gathering the necessary evidences, they produce an exact copy of the evidence, called the forensics copy, rather than the original copy. This is done to avoid the destruction of original evidence. During the course of the investigation, the evidences may go through several phases. At any point of time, the investigator must be completely sure that the evidence is the exact same copy of the forensics copy. Hash functions play a very crucial role in this regard. They are used to verify whether the evidence has been subject to some unauthorized manipulation. SHA-1 and SHA-2 has been among the widely used hash functions. But nowadays, SHA-1 is considered weak. Furthermore, although SHA-3 is more secure than its earlier counterparts, it is considered to be slow on general purpose processors and as such, they are not so popular as yet. In this context, this assignment proposes a simple, but effective mechanism to fulfill the needs of cyber forensics using the combined capabilities of hashing functions, along with other cryptography concepts such as SALT.**

***Index Terms*—Hashing, SHA-1, SHA-2, SHA-3, Cyber forensics, SALT, Cryptography**

I. Introduction

A. Phases in digital forensics

Digital forensics entails following phases:

1) Identification
2) Preservation
3) Analysis
4) Documentation
5) Preservation

Out of these, preservation and analysis are the phases wherein hash functions are used.

During the preservation phase, the digital evidence is isolated, secured and preserved. In this phase, hashing function is used to compute the hash value of the evidence.

During the Analysis phase, hashing algorithm is used to check the integrity and completeness of the forensic evidence that was procured during the preservation phase.

B. Hash functions

Hash functions are algorithms that maps the input text of arbitrary length to a fixed length hash value. The main i d e a behind using hash functions in verifying the integrity of data and completeness is that, the hash value produced by a hash function is unique for a given input text. Even the slightest variation in the input text can produce a different hash value [1]

Any good hash function should satisfy two important properties as can be seen from [2]. They are as follows,

1) Pre-image resistant
2) Collision resistant

Creation of hash functions involves some complicated mathematics as in [3]. Furthermore, to design a secure hash function, that can satisfy collision resistance, can be mathematically challenging. If the input length is sufficiently large and the hash value is fixed in size, it is quite possible that multiple inputs may share common hash values [4]

Standard approach to design hash functions, as proposed by Merkle and Damgard is – to construct a compression function and iterate it. They also proved that – if the compression function is collision resistant then, the resultant hash function will also be collision resistant [5]

More than a decade ago, cryptography experts discovered collisions in MD5 algorithm. And later on, MD5 algorithm was truly broken. Even though, SHA-1 and SHA-2 are considered to be more complex as compared to MD5, they came from the same family as MD5. Therefore, the techniques that were employed to compromise MD5 can be employed on SHA- 1 and SHA-2 also [6], [7], [8], [9]. Furthermore, SHA-3 is not widely used due to its performance on general purpose processors [10], [11]

In this assignment, two hash functions - SHA-1 and SHA-2, are combined along with the use of SALT, to compute collision resistant hash values.

C. SALT

Small random data, typically about 4 bytes in size, is referred to as SALT. Traditionally, security practitioners have been using SALT as an additional input to hash functions, to safeguard passwords/passphrases in storage. In other words, generally, SALT is used to protect user's passwords from pre- computed hash attacks such as rainbow tables.

But in this assignment, employed SALT for a different purpose. It is to make our hash functions resistant to collisions. The details regarding the use of SALT, along with hash functions is described elaborately in the following section.

II. Proposed model

The idea here is to combine the capabilities of both SHA-1 and SHA-2, along with the use of SALT

SHA-1 takes input of arbitrary length and produces 128-bit hash value. There are instances where collisions in SHA-1 has

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:56 UTC from IEEE Xplore. Restrictions apply.

occurred and as such, it is considered to be weak nowadays. Likewise, SHA-2 accepts input of arbitrary length and produces 256-bit hash value. But, thus far, there hasn't been any instance where collisions have occurred in SHA-2.

As a result, considered SHA-2 to be more secure as compared to SHA-1.

A. Working principle

Assume a sample data whose size is in the order of gigabytes. And, required to compute the hash value of such a data.

Hash values tend to compress the data to a fixed size. And whenever a large data is compressed, it is possible that multiple inputs share the same hash values. In other words, there may be collisions. Consider the following equation,

$$X = SHA1(Inputdata + salt1) \quad (1)$$

This equation by itself is not very significant. That is, simply by the introduction of salt1 in the above equation, does not make it resistant to collisions.

Now, consider another equation which is as follows,

$$H = SHA2(X + salt2) \quad (2)$$

Equation (2) provides a couple of benefits. They are as follows,

1) The intermediate hash value obtained in equation (1) is supplied to SHA2, along with salt2 in equation (2)
2) Although, equation (1) may produce the same hash value for some other input data, salt2 in equation (2) will guard against further collisions from occurring.
3) Because, SHA-2 does not compress the data but expands. As a result, there can be no collisions resulting from equation (2)

Consider the following illustration,

$$X1 = SHA1(inputdata1 + salt1A)$$
$$X2 = SHA1(inputdata2 + salt1B)$$
$$Here, X1 = X2 \text{ is a possibility}$$

$$Assume, X1 = X2$$

$$H1 = SHA2(X1 + salt2A)$$
$$H2 = SHA2(X2 + salt2B)$$
$$H1 \text{ not equal to } H2$$

In the above illustration, H1 cannot be equal to H2 for following reasons,

1) Even if X1 = X2, salt2A and salt2B are going to be different
2) Moreover, SHA2 does not compress the input data. But rather, it inflates the input data into 256 bits

The working of proposed hashing scheme is also illustrated in the following block diagram

B. Design

In the context of digital forensics, hashing is used in preservation and analysis phases. Generally, forensic tools are used to compute hash values of digital evidences. A high-level block diagram illustrating the computation of hash value using forensics tool is shown below,

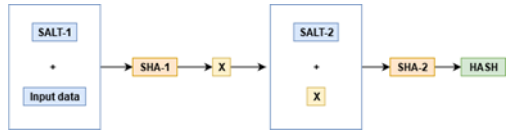


Fig. 1. Input data is combined with salt1 before inputting to SHA-1. Intermediate hash value, x is combined with salt2 to produce the final hash value

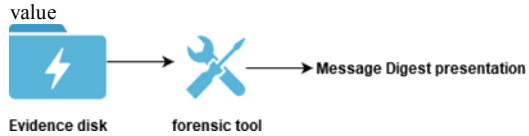


Fig. 2. Computation of hash value using forensics tool

If were to visualize a software that is developed based on the proposed hashing scheme, its workflow would be as follows,

When the investigator runs the software program, it prompts him to choose from the menu,

1) Compute hash for new case (Preservation phase)
2) Validate a case (Analysis phase)

During the preservation phase:

1) Investigator starts the program
2) Program prompts the investigator to enter a new case-id
3) Investigator enters the case-id and taps on enter key
4) Program prompts the user to provide the path to evidence data, whose hash it needs to **compute**
5) Investigator provides the path to evidence data
6) Program accepts the input data and
   a) Generates random salt value
   b) Appends the salt value to the input data
   c) Computes hash value of the input data
   d) Stores the computed hash and salt values against the input case-id, in its internal database
7) Displays the computed hash value to the user

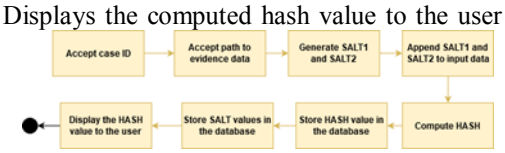


Fig. 3. Forensics tool workflow — Preservation phase

During the analysis phase:

1) Investigator starts the program
2) Program prompts the user to enter existing case-id
3) Investigator enters the case-id and taps on enter key
4) Program prompts the user to provide the path to evidence data, whose hash it needs to **validate**
5) Investigator provides the path to evidence data
6) Program accepts the input data and retrieves the following from the database
   a) Salt values

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:56 UTC from IEEE Xplore. Restrictions apply.

b) Hash value computed during the preservation phase

Program computes the hash value of the input data using

7) the salt values retrieved from the database
8) Program compares the newly computed hash value with the hash value that was computed during the preservation phase
9) Reports the output to the Investigator as any one of the following,
   a) Data is not tampered
   b) Data has been tampered

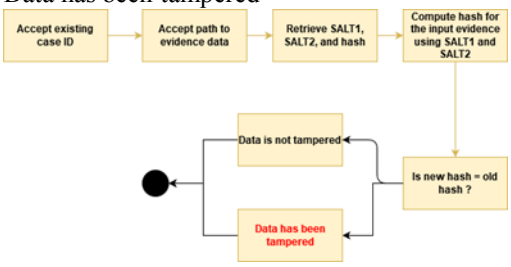


Fig. 4. Forensics tool workflow — Analysis phase

III. Related Work

Early algorithms sought to solve the problem of collision by complex mathematics as in [3]

Traditionally, concept of SALT was used only to safeguard user passwords against rainbow table attacks. But, in this assignment, have proposed to use SALT with hash functions for a different cause. That is, to avoid possible collisions. It suggests that the idea of using SALT with hash functions can be extended to other areas as well. And particularly, in the area of digital forensics where the input data is usually huge. And besides, digital forensics is open for several opportunities as in [12]

Furthermore, since have combined two different hash functions (SHA-1 and SHA-2) in our proposed design, it may appear to be an overhead on the computing device. But contrary to our belief, it impacts of using two hashing functions is not so significant for the following reasons,

1) In the context of digital forensics, typically, the size of the data will be in the order of several gigabytes. For input sizes such as this, the time complexity of SHA-1 can be simplified as O(n). Where, n is the input length.
2) Regardless of what the input size is, the input to the outer SHA-2 is always fixed. That is, 128-bit hash produced by SHA-1 + 32 bits of salt value. This makes the time complexity of SHA-2 always a constant, the overall complexity of the hashing scheme will continue remain O(n)

Also, one might as well use SHA-2 and MD5 combination, depending upon the requirement. But the basic rules for choosing hash functions are as follows,

1) The outer hash function should be more secure as compared to the inner hash function
2) The inner hash function may expand or compress the input data. But the outer hashing function should always inflate the input data

The general idea can be expressed as follows,

Moresecrehash (*lesssecurehash* (*inputdata* ✚ *salt*1) ✚ *salt*2)

In the above equation, more secure hash is the inflator and less secure hash is usually the compressor in the context of digital forensics.

IV. Result Analysis

A. Digital forensics

Any form of digital evidence – may it be files on a computer, hard disk drive, pen drive, etc. are mostly on a file system.

So, in this assignment, consider few files on the Linux file system as a piece of evidence.

Assume that on a certain file system, there are three files – text file having a log of browser history, user's medical report, and a secret note. And, this text file contains some confidential notes.

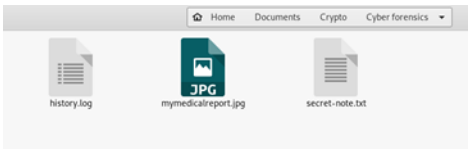


Fig. 5. Consider a file system having 3 important files

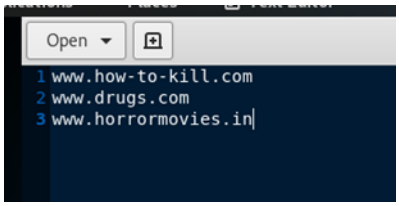


Fig. 6. A file containing the suspect's browser history logs

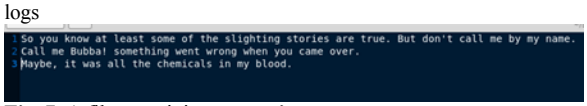


Fig. 7. A file containing suspect's secret note

Fig. 8. An image file containing the suspect's recent medical report

When the entire directory encompassing all the files under consideration, is provided to the hashing algorithm, it produces a 256-bit hash value that represents the entire data set.

During the Preservation phase, the algorithm outputs the hash value, and also the stores the salt values.

For example,

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:56 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. Hash and salt values computed during the preservation phase

Hash value	Salt1	Salt2
3565f3c79951e256831fffae16500e9785e0a5771f0997eb01177d4eaedb0752	!#zh	(34&

The values of SALT-1 and SALT-2 are recorded for later use. Later during the Analysis phase, the investigator is required to conduct the autopsy of the forensics data. But before that, he / she has to check the validity of the evidence. Therefore, they compute the hash value of the file system once again.

This time, along with the directory, the program requires the previously computed SALT values. The algorithm can then recompute the hash value for a given directory.

TABLE 2. Hash value computed during the analysis phase

Hash value	Salt1	Salt2
e70171386b79a16850c29a529f730e42e2673f99831d5edf3e2d61fb2970a24f	!#zh	(34&

Notice the change in the hash values depicted in table 1 and 2. Even though the salt values are the same, there seems to be a change in the hash value. This means that the evidence has been tampered by some unauthorized party.

However, if the hash values were to be the same, then it would mean that the data under consideration is a valid piece of evidence and as such, it has not undergone any unscrupulous manipulation.

B. Analysis of the hash function

Analysis of algorithms on various computing devices is quite a challenging task. Here, an attempt has been made to analyze the proposed scheme of hashing on a device with following specifications,

TABLE 3. Specifications of the computing device on which the analysis was performed

Input data (GB)	1.20	6.71	38.4	105
Time (s)	29.293	64.334	385.759	1031.762

The performance of the proposed hashing function was determined by recording the time taken to compute the hash for increasing input size. Please see the below table for details,

TABLE 4. Time analysis of the proposed algorithm

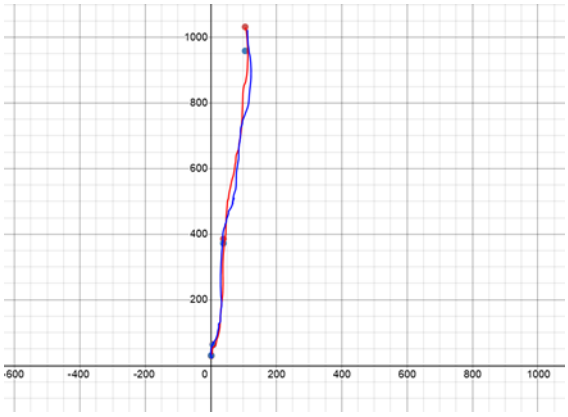
Component	Specification
Computer	Intel i7 6th gen
Logical processors	4
Cores	2
Memory	16 GB
L1 Cache	128KB
L2 Cache	512 KB
L3 Cache	4 MB

In order to check if the proposed scheme is causing over-head on the computing device, the experiment was repeated on the same data set by considering SHA-2 alone. The results were as follows,

TABLE 5. Time analysis of SHA-2 hash function

Input data (GB)	1.20	6.71	38.4	105
Time (s)	30.484	62.733	372.358	958.798

As can be seen from the result, there is no significant change in time. Both the hashing functions are growing linearly in time with growing input size.



Graph 1. Graph depicting the comparison between the proposed hashing scheme (blue) v/s SHA-2 (red). As can be seen, there is no significant change in terms of time.

V. CONCLUSION

This assignment offered an insight about the significance of hash functions in the field of digital forensics. It helped us understand the power of hash functions in terms of detecting even the smallest of the changes in the input.

Furthermore, it helped us understand the viewpoints of various researchers and cryptography experts, on the use of some hashing functions such as MD5.

Some cryptography experts have insisted people to not use MD5 anymore, for the fact that it is not collision resistant and can be subjected to many attacks such as – Birthday attack, Rainbow attack, etc.

Yet, there are others who assert that the occurrence of collision resistance in the real time is very rare and as such, thinking over that is a waste of time.

But it is always safe to explore, learn and migrate to a newer, more secure and a safer algorithm.

VI. REFERENCES

[1] J. S. Coron, Y. Dodis, C. Malinaud, and P. Puniya, 2005.
[2] S. S. Thomsen, 2005.
[3] T. Ristenpart and T. Shripmton, 2004.
[4] S. Contini, A. Lenstra, and R. Steinfeld, 2006.
[5] "Isaac Newton Institute workshop," 1996. [6] D. Hans, 1996.
[7] Dobbertin Hans. "Cryptanalysis of MD5 compress." German Information Security Agency; May 1996.
[8] X. Wang and Y.L. Yin and H. Yu.Finding Collisions in the full SHA-1. To appearin Advances in Cryptology –Crypto'05.
[9] E. Biham and R. Chen, 2004.
[10] E. Kavun and T. Yalcin, 2012.
[11] S. Chang, M. Lee, M. Nandi, and Yung, 2006.
[12] M. Chernyshev, S. Zeadally, Z. Baig, and A. Woodward, "Mobile Forensics: Advances, Challenges, and Research Opportunities," pp. 42–51, 2017. Available: 10.1109/msp.2017.4251107; https://dx.doi.org/10.1109/msp.2017.425110

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:48:56 UTC from IEEE Xplore. Restrictions apply.